AI Agent 辅助 MXFP8 GEMM Kernel 优化报告

目录 1. 背景 2. 性能结果 3. 优化历程 4. 关键优化技术 5. 失败尝试与教训 6. AI Agent 辅助优化工作模式

1. 背景

在 AMD MI355X (GFX950 / CDNA4) 上开发高性能 MXFP8 GEMM kernel，目标是接近 hipBLASLt FP8 (tensorwise) 的性能水平。

- **架构**: GFX950, 256 CU, HBM3
- **数据格式**: A/B = FP8 (e4m3), Scale = MXFP8 (per-32-element), C = FP16, 累加器 = FP32
- **分块**: 256×256×128, MFMA 16×16×128, 4 waves/workgroup, 256 threads

经过多轮人工优化，kernel 性能已进入瓶颈区 — 剩余优化空间集中在指令调度和硬件微架构层面。由于 hipcc 编译器无法总是生成最优的指令排列，需要对照编译产出的汇编来验证优化是否生效、定位编译器引入的额外开销。每次改动都涉及分析大量汇编、运行 profiling、反复试错，工作量大且重复性高，人工难以在短时间内系统性地完成。因此我们引入 AI Agent 辅助优化，由人工把控方向，Agent 负责分析和执行。

2. 性能结果

2.1 算子性能

矩阵尺寸	hipBLASLt FP8	hipBLASLt MXFP8	Baseline	优化后	vs hipBLASLt FP8
4096^3	2525	1491	2291	**2531**	**100.2%**
8192^3	3195	1512	2643	**2991**	93.6%
16384^3	3261	1231	2673	**3029**	92.9%

> 单位: TFLOPS。hipBLASLt FP8 (tensorwise scale) 作为性能参考上界；hipBLASLt MXFP8 作为同格式对比基线。

2.2 Primus-Turbo Benchmark (量化 + MXFP8 GEMM)

将优化后的 kernel 集成到 Primus-Turbo（含量化 + GEMM），benchmark 结果：

Hardware	Backend	Avg Forward TFLOPS	Avg Backward TFLOPS
MI355X	hipBLASLt	1132	1179
MI355X	**Primus-Turbo (本工作)**	**1795**	**1977**
GB200	TE	1947	1864

Forward 性能较 hipBLASLt 提升 **58.6%**；Backward 超越 GB200 TE。

3. 优化历程

3.1 优化方法

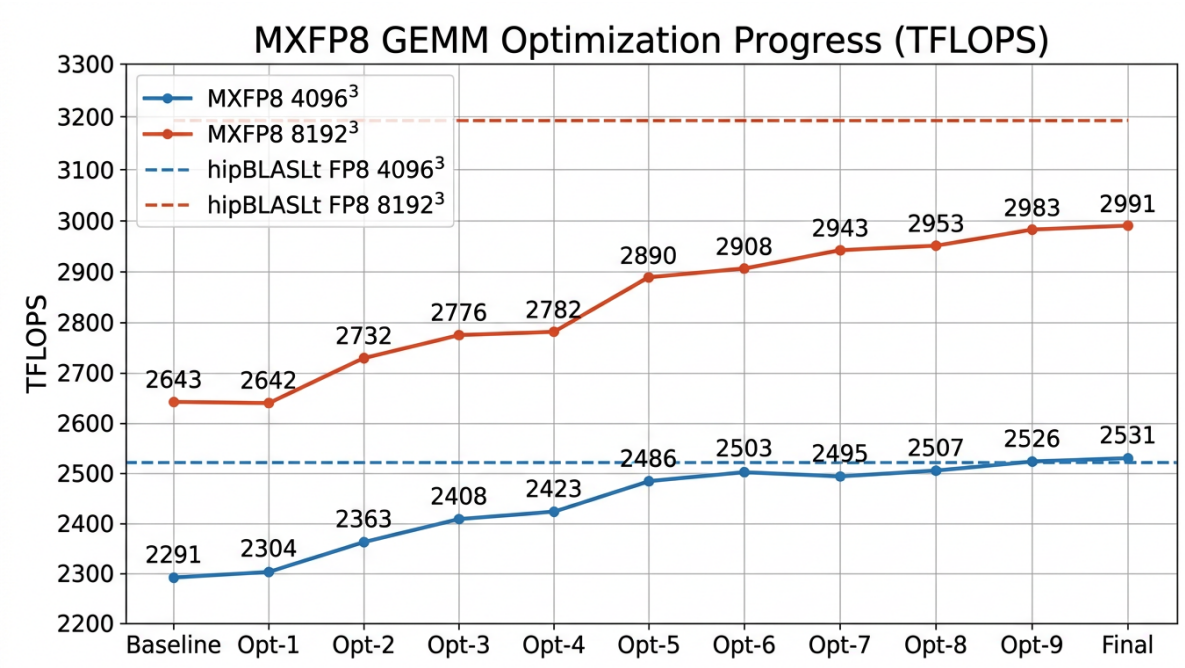
整个优化分为两个阶段:

- **Baseline (人工优化)**: 经历 4 个架构版本迭代，从零开始构建 MXFP8 GEMM kernel。
- V1 — 加入 MXFP8 scale 支持
- V2 — 细粒度流水线 + SRD 预构建
- V3 (Baseline) — 引入 Pinned Register 架构（参考 HipKittens 思路）+ 零 mid-barrier 流水线
- 4K: 2205 → 2291 T，8K: 2526 → 2643 T
- **Baseline → 最终版 (人工引导 + AI Agent 协同)**: 受 AVO 方法论启发，由人工提出优化方向和约束，AI Agent 完成汇编分析、方案实现、性能测试和迭代调优，累计提升 **+11.9%**。

3.2 优化步骤

#	阶段	4096^3	8192^3	16384^3	累计提升
0	Baseline (人工优化)	2291	2643	2673	—
1	+同步指令替换 (QW)	2304	2642	—	+0.3%
2	+SGPR 预计算 soff	2363	2732	—	+3.3%
3	+SGPR 预计算 m0	2408	2776	—	+5.1%
4	+减少 sched_barrier	2423	2782	—	+5.6%
5	+SpreadLDG (显存请求均匀分散)	2486	2890	—	+8.9%
6	+数据预取前移	2503	2908	—	+9.7%
7	+自适应 WGM + 去 sched_barrier	2495	2943	2950	+10.2%
8	+Epilogue SpreadLDG	2507	2953	—	+10.7%

#	阶段	4096^3	8192^3	16384^3	累计提升
9	+循环不变量外提 + 代码清理	2526	2983	—	+11.7%
10	**+统一 SRD (最终版)**	**2531**	**2991**	**3029**	**+11.9%**



4. 关键优化技术

4.1 SGPR 预计算 (+5.1%)

问题: 主循环中有 36 条地址转换指令 (VGPR→SGPR)，占用计算单元，且每条后面都跟一个空等待指令。

方案: 将所有地址在循环外预计算到标量寄存器中，循环内只用纯标量运算切换地址。

结果: 地址转换指令 36 → 4 条 (-89%)，空等待指令 24 → 0 条，主循环总指令 -19%。

4.2 SpreadLDG — 显存请求均匀分散 (+4.0%)

问题: 每个计算阶段的 6 条显存加载请求集中发射，导致 L2 cache 竞争。

方案: 将 6 条请求分散到计算指令间隙中，均匀分布。

结果: L2 命中率 74% → 78.5%，DRAM 流量 -24%，MFMA 利用率 52.3% → 54.2%。

4.3 数据预取前移 (+0.9%)

将每一轮的数据加载提前一个阶段发起（计算第 N 轮时加载第 N+2 轮），延迟隐藏窗口增大 67%。

4.4 自适应 Tile 分组 (+1.4% on 16K)

根据矩阵 N 维度大小动态选择 Tile 分组策略，大矩阵用更方的分组以提高 B 矩阵 cache 复用。16K 性能 +10.4%。

4.5 编译器协作优化 + 代码清理 (+1.7%)

移除手工调度约束，让编译器自由调度；外提循环不变量，编译器反而找到了更优的指令排列。

5. 失败尝试与教训

#	优化方向	结果	教训
1	显存加载移到 MFMA+LDS 区域 (3 次尝试)	-2~-4%	buffer_load_lds 和 ds_read 共用 LDS 端口，不能同时发射
2	NT store for C (绕过 L2)	**-18%**	global_store_short 加 NT flag 走了更慢的路径
3	LLC-aware swizzle (CHUNK=32)	8K±0%, 部分 shape 暴跌	固定 CHUNK 破坏非整除 shape 的 L2 局部性
4	vmcnt 调优 (3/9/15/18)	vmcnt(6) 最优	更多在途 load 不一定帮助，vmcnt(6) 是最佳平衡点
5	s_setprio / launch_bounds(256,2)	无变化	occupancy=1 下硬件无法 wave 切换隐藏 stall

6. AI Agent 辅助优化工作模式

6.1 背景

AVO (Agentic Variation Operators) 是 NVIDIA 提出的 AI 自主 kernel 优化方法 (arXiv: 2603.24517)，在 Blackwell B200 上全自动运行 7 天，优化 Attention kernel 超越 cuDNN 3.5%、FlashAttention-4 10.5%。

6.2 我们的实践

我们最初尝试了全自动模式，但 review 工作记录后发现几个问题： - **搜索空间过大**: Agent 会花大量时间探索人工已经可以排除的方向（如调整分块大小、增加 occupancy），消耗大量计算资源但没有产出 - **方向判断不准**: Agent 有时否定正确的方向 — 第一版实现性能下降就直接回退，没有深入排查是实现问题还是方向本身不对 - **高基线下优化更难**: Baseline 已经较高，剩余优化空间小，需要对硬件行为有更精准的判断，全自动容易走弯路

因此调整为**人工引导 + Agent 执行**的协作模式：

步骤	人工	AI Agent
1. 代码 + 汇编 + Profiling 分析	粗读关键段，指出关注区域	阅读 HIP C++ 源码理解逻辑，分析编译器生成的汇编定位指令瓶颈，运行 rocprof-compute 采集硬件性能数据，综合总结瓶颈
2. 确定优化方向	Review Agent 的分析结论，排除不可行方向，补充自己的优化思路（如流水线重组），设定约束条件	基于分析结果提出候选优化点供人工筛选
3. 方案实现	Review 关键改动，判断是否符合预期	编写 HIP C++ 代码、修改 inline assembly、调整流水线结构
4. 测试验证	—	编译、运行正确性测试、采集多个 shape 的 TFLOPS
5. 结果判断	决定保留还是回退：区分"方向对但实现有问题"和"方向本身不对"	提供数据支撑
↩ 迭代	回到步骤 1，进入下一轮	

典型流程 (以 SpreadLDG +4.0% 为例)：Agent profiling 发现 L2 stall 高并提出候选优化点 → 人工确认"显存请求应该分散" → Agent 分析汇编定位集中发射点 → Agent 实现分散方案 → Agent 测试 +4.0% → 人工确认保留。

6.3 后续计划

将本次积累的经验整理为结构化 skill，逐步提高 Agent 自主判断能力，向全自动模式靠拢。

测试环境: AMD MI355X, ROCm 7.1.0